EXP 6	A PYTHON PROGRAM TO IMPLEMENT THE GRADIENT DESCENT METHOD

AIM:

To iteratively adjust the parameters of a model in order to minimize the error between the

predicted and actual values by using the Stochastic Gradient Descent (SGD) algorithm.

SGD updates the model parameters based on each training example, allowing for faster

convergence and handling larger datasets efficiently compared to batch gradient descent.

PROCEDURE:

1. INITIALIZATION: • We initialize the weights to zeros. • X is the feature matrix, and y is the target vector. • Learning rate controls the step size of each update. • Epochs are the number of iterations over the entire dataset.

2. GRADIENT COMPUTATION:

• For each sample in the dataset, we compute the gradient of the loss with respect

to the weights. • The gradient is calculated as (prediction - target) * feature.

3. WEIGHTS UPDATE: • We update the weights by moving them in the opposite direction of the

gradient, scaled by the learning rate.

PROBLEM 1:

To apply **Stochastic Gradient Descent (SGD)** to a simple linear regression model and find the optimal weights.

CODE:

```
import numpy as np
def stochastic_gradient_descent(X, y, learning_rate=0.02, epochs=800):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)
    for epoch in range(epochs):
        for i in range(n_samples):
            prediction = np.dot(X[i], weights)
            error = prediction - y[i]
            gradient = error * X[i]
            weights -= learning_rate * gradien
    return weights
if __name__ == "__main__":
    np.random.seed(1)
    X = 3 * np.random.rand(100, 1)
    y = 5 + 2 * X + np.random.randn(100, 1)
    X_b = np.c_[np.ones((100, 1)), X]   # Add bias
    y = y.ravel()
    weights = stochastic_gradient_descent(X_b, y)
    print("Weights:", weights)
```

OUTPUT:

Weights: [5.12 1.95]

PROBLEM 2: To find the minimum of a quadratic function using Gradient Descent.

CODE:

```
import numpy as np
def quadratic_loss(x):
    return x**2 + 4*x + 5
def quadratic_gradient(x):
    return 2*x + 4
# Initialize
x = np.random.randn()
lr = 0.1
# Gradient Descent
for _ in range(60):
    x -= lr * quadratic_gradient(x)
print("Minimum of quadratic function:", x)
```

OUTPUT:

Minimum of quadratic function: -1.99998

RESULT:

Thus, solving any Gradient Descent Method using the Stochastic Gradient Descent

Algorithm in python has been implemented and executed successfully.